



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

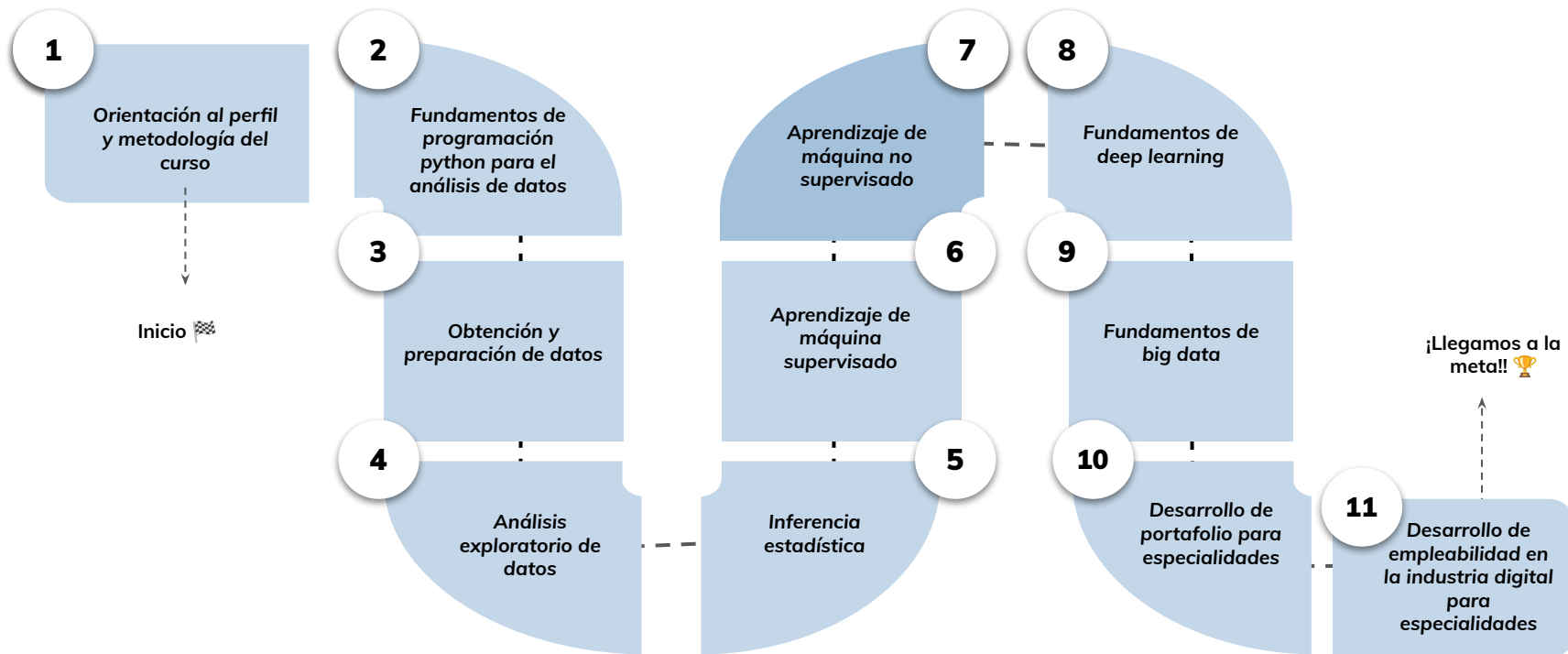


› Algoritmos de clusterización- › Parte I

Aprendizaje Esperado 4: Implementar un modelo predictivo aplicando algoritmos de clusterización utilizando lenguaje python para resolver un problema.

Hoja de ruta

¿Cuáles skills conforman el programa? Fundamentos de Ciencia de Datos



Roadmap de lecciones

¿Cuáles *lecciones* estaremos estudiando en este módulo?

APRENDIZAJE DE MÁQUINA
NO SUPERVISADO

1

REDUCCIÓN
DIMENSIONAL

3

TÉCNICAS DE REDUCCIÓN
DIMENSIONAL

5

CLUSTERIZACIÓN Y SUS
PRINCIPALES ALGORITMOS

ALGORITMOS DE
CLUSTERIZACIÓN

Learning Path

¿Cuáles temas trabajaremos hoy?

4.

Implementación de
algoritmos de reducción
de dimensionalidad

Exploración práctica del uso de técnicas como PCA y t-SNE para mejorar la eficiencia y comprensión de los datos en contextos reales.

Aplicaciones y beneficios

Visualización de agrupamientos y detección de patrones

Reducción de ruido y mejora del rendimiento de modelos

Implementación y evaluación

Buenas prácticas: preprocesamiento, selección de componentes

Métricas para evaluar la calidad de la reducción



Objetivos de aprendizaje

¿Qué aprenderás?



- Implementar algoritmos de clusterización en Python paso a paso
- Aplicar K-Means, agrupamiento jerárquico y DBSCAN sobre un dataset
- Evaluar el rendimiento de cada modelo mediante visualizaciones y métricas
- Analizar cómo elegir el número óptimo de clústeres (método del codo, silueta)
- Interpretar los resultados para resolver un problema real

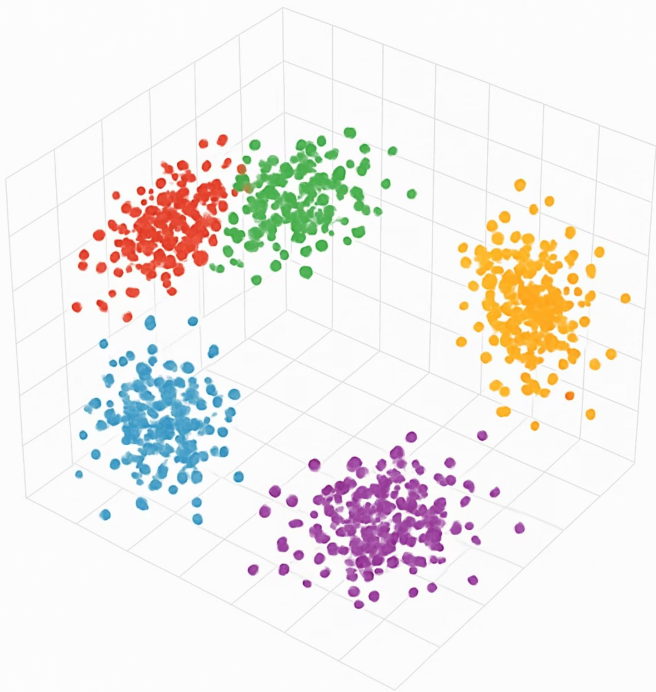
Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Comprendimos cómo utilizar técnicas de reducción de dimensionalidad para abordar situaciones del mundo real
- Identificamos cómo estas técnicas contribuyen a mejorar tanto la visualización como la eficiencia de los modelos
- Estudiamos los pasos esenciales para su correcta implementación: normalización, elección de componentes y validación
- Investigamos diversas métricas que permiten evaluar la calidad de la representación reducida

Algoritmos de clusterización



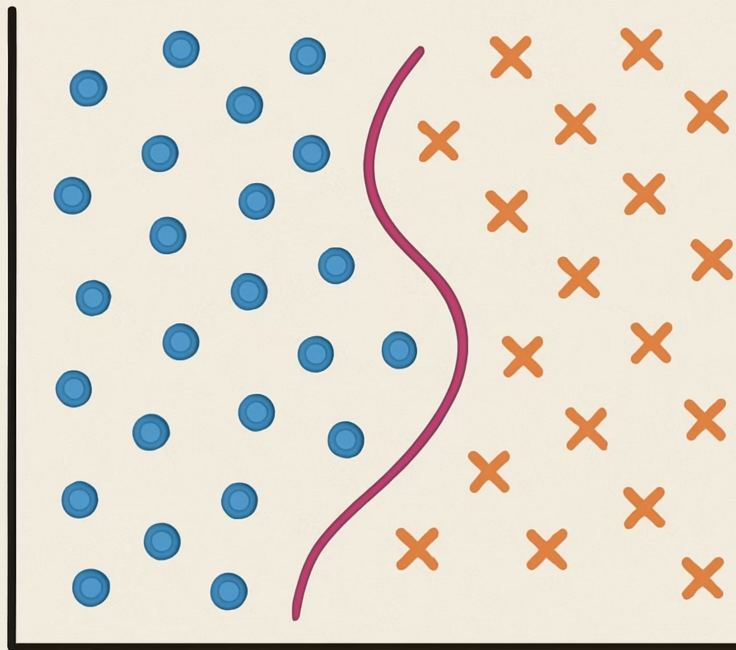
Algoritmos de Clusterización en Machine Learning

El aprendizaje no supervisado es una disciplina clave en el campo del Machine Learning, y dentro de ella, la clusterización ocupa un lugar fundamental. Esta técnica permite agrupar datos sin etiquetas previas, basándose únicamente en similitudes o distancias entre observaciones. Su utilidad se extiende a distintos campos como la biología, el marketing, la astronomía y la informática.

Introducción a la Clusterización

Los algoritmos de clusterización ayudan a descubrir patrones ocultos, a clasificar automáticamente elementos y a reducir la complejidad de los datos. En esta lección nos enfocaremos en los principales algoritmos de agrupamiento disponibles, su implementación en Python, y su aplicabilidad a problemas reales.

MACHINE LEARNING PATTERN RECOGNITION IN DATA



Principales Algoritmos a Estudiar



K-Means

Algoritmo basado en centroides que agrupa datos en K clústeres predefinidos.



Agrupamiento Jerárquico

Crea una estructura en forma de árbol que representa relaciones entre observaciones.



DBSCAN

Algoritmo basado en densidad para encontrar estructuras arbitrarias y detectar outliers.

Analizaremos con profundidad K-Means, agrupamiento jerárquico y DBSCAN. Cada uno con sus particularidades, ventajas y desventajas. También aprenderás cómo elegir el número óptimo de clusters y evaluar los resultados obtenidos a través de visualizaciones y métricas.

¿Qué es la Clusterización?

La clusterización, o agrupamiento, es una técnica de análisis exploratorio que permite identificar grupos o segmentos en un conjunto de datos sin necesidad de conocer etiquetas previas. Se basa en la similitud de características y busca agrupar observaciones que se parezcan entre sí y estén lo más alejadas posible de otros grupos.

Aplicaciones de la Clusterización

Segmentación de Clientes

Agrupa consumidores con comportamientos similares para estrategias de marketing personalizadas.

Agrupación de Documentos

Organiza textos por temáticas o similitud para facilitar búsquedas y análisis.

Detección de Anomalías

Identifica valores atípicos que no pertenecen a ningún grupo establecido.

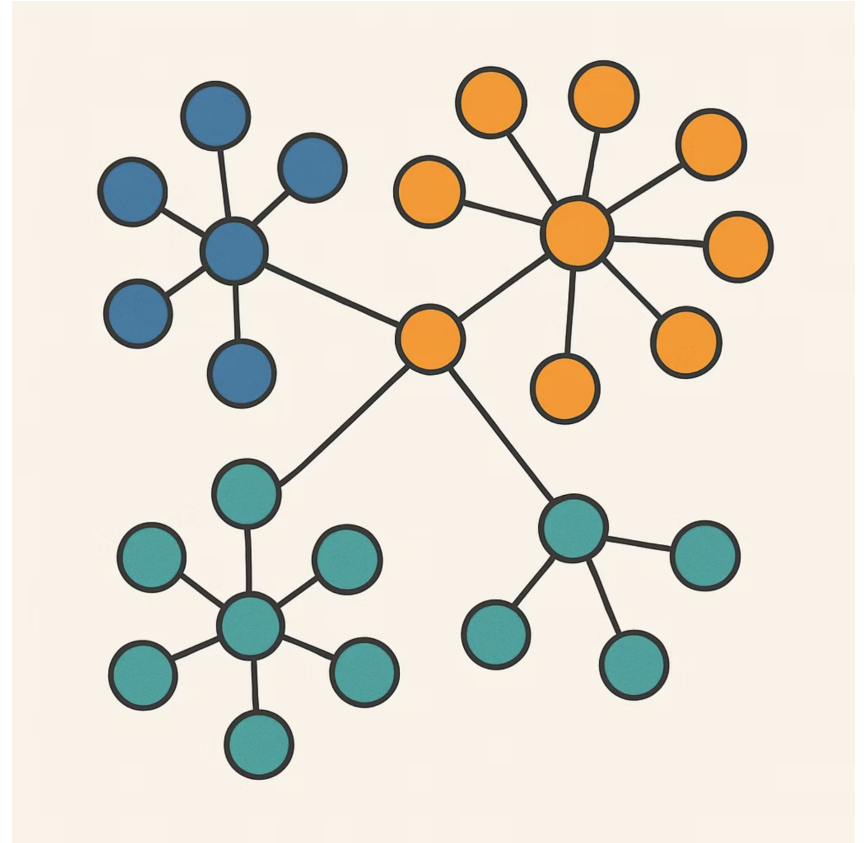
Reducción de Ruido

Filtra datos irrelevantes para mejorar la calidad del análisis.

Esta técnica es utilizada en una amplia gama de aplicaciones como segmentación de clientes, agrupación de documentos, detección de anomalías, reducción de ruido y compresión de datos. Su flexibilidad la hace ideal para descubrir patrones cuando no hay una salida definida.

Utilidad de la Clusterización

El resultado de una clusterización no supervisada puede usarse para comprender mejor la estructura de los datos, hacer recomendaciones personalizadas o generar etiquetas automáticas que luego alimenten algoritmos supervisados.

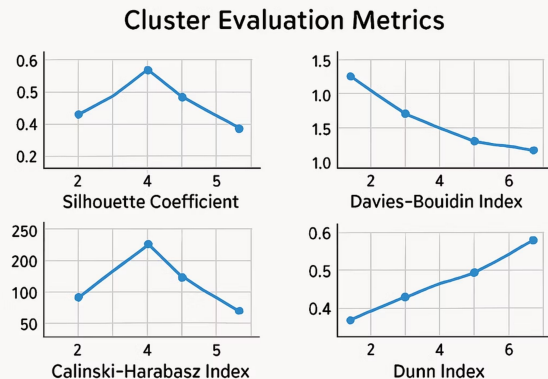


Desafíos en la Clusterización

- ❑ Una de las dificultades comunes al aplicar clusterización es definir el número óptimo de grupos (clusters) y seleccionar el algoritmo adecuado según la forma, densidad y ruido en los datos.

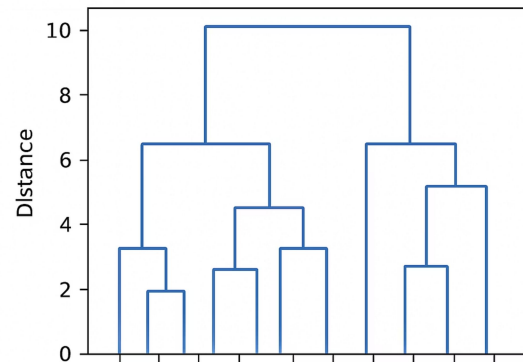
Evaluación de Resultados

Para evaluar el resultado de un modelo de agrupamiento, se suelen utilizar medidas como el coeficiente de silueta, la cohesión intra-cluster y la separación inter-cluster.



Agrupamiento Jerárquico

El agrupamiento jerárquico crea una estructura en forma de árbol que representa las relaciones entre observaciones. Este árbol se conoce como dendrograma. Existen dos tipos principales de agrupamiento jerárquico: aglomerativo y divisivo.



Tipos de Agrupamiento Jerárquico

Aglomerativo

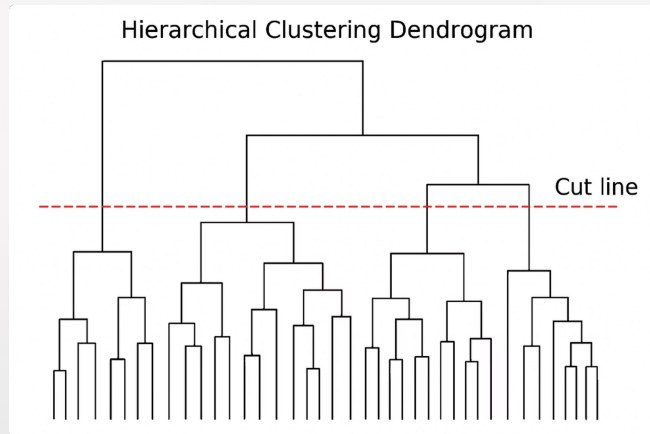
Parte con cada punto como un clúster individual y va fusionando los más cercanos hasta agruparlos en uno solo.

Divisivo

Inicia con todos los puntos en un solo grupo y los va dividiendo progresivamente.

Dendrograma

El dendrograma ayuda a decidir visualmente el número óptimo de clusters. Para aplicar este tipo de agrupamiento se siguen varias etapas:



Etapas del Agrupamiento Jerárquico

Normalización de datos

Preparación y estandarización de las variables para el análisis.

Aplicación del algoritmo de fusión

Uso de métodos como enlace simple, completo o promedio para agrupar.

Cálculo de la matriz de distancias

Determinación de la similitud entre cada par de observaciones.

Visualización del dendrograma

Representación gráfica de la estructura jerárquica resultante.

Ventajas del Agrupamiento Jerárquico



No requiere definir el número de clusters

El dendrograma permite decidir el número óptimo de grupos después de ejecutar el algoritmo.



Proporciona una vista jerárquica clara

Muestra relaciones anidadas entre grupos, facilitando la interpretación de la estructura de los datos.

Desventajas del Agrupamiento Jerárquico



Escalabilidad limitada en grandes conjuntos

El cálculo de la matriz de distancias puede ser computacionalmente costoso para conjuntos de datos extensos.



Alta sensibilidad al ruido y outliers

Los valores atípicos pueden distorsionar significativamente la estructura del dendrograma.

Implementación en Python

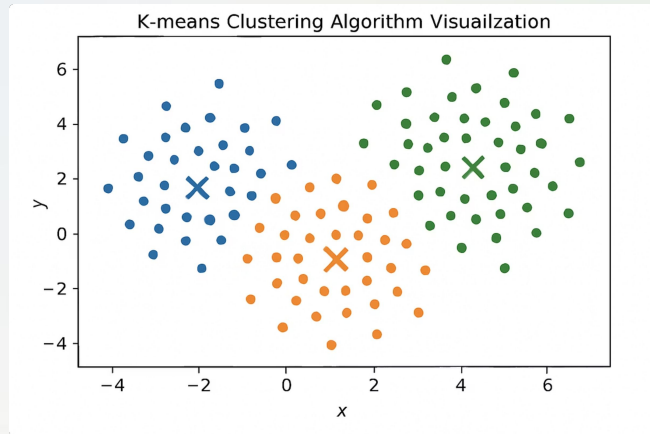
```
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt
import pandas as pd

# Datos de ejemplo
X = pd.read_csv("datos.csv")
Z = linkage(X, 'ward')

plt.figure(figsize=(10, 7))
dendrogram(Z)
plt.title("Dendrograma")
plt.xlabel("Observaciones")
plt.ylabel("Distancia")
plt.show()
```

Algoritmo K-Means

K-Means es un algoritmo que agrupa datos en K clústeres definidos por sus centroides. El proceso es iterativo y busca minimizar la varianza dentro de los grupos.



Etapas del Algoritmo

K-Means

1

Elegir K

Definir la cantidad de grupos en los que se dividirán los datos.

2

Inicializar los centroides

Seleccionar aleatoriamente K puntos como centroides iniciales.

3

Asignar puntos

Asignar cada punto al centroide más cercano según una medida de distancia.

4

Recalcular centroides

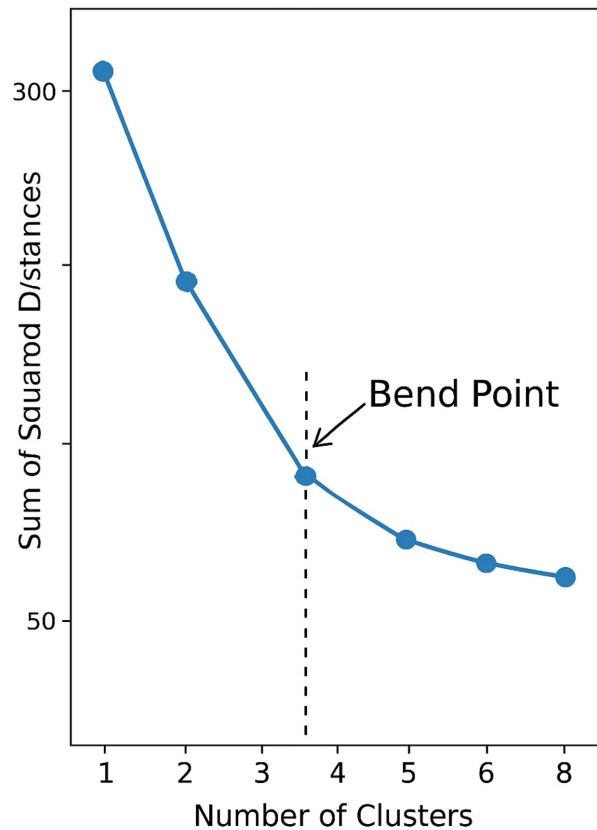
Actualizar la posición de cada centroide como el promedio de los puntos asignados.

5

Repetir hasta convergencia

Repetir los pasos 3 y 4 hasta que no haya cambios significativos en los centroides.

Eibow Method



Selección del Valor de K

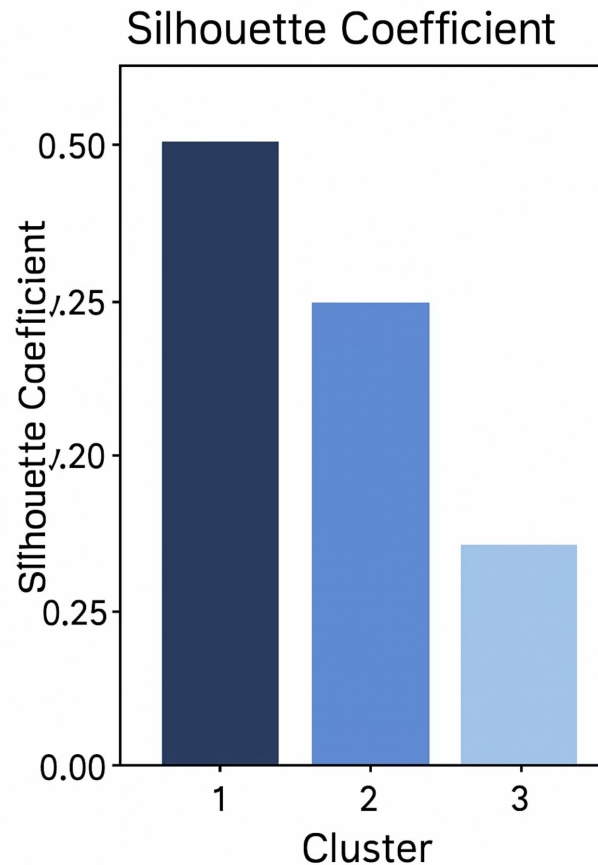
Método del Codo

El método del codo es una técnica visual para determinar el número óptimo de clusters en K-Means. Consiste en graficar la suma de errores cuadráticos (WSS) para diferentes valores de K y buscar el "codo" en la curva.

Evaluación del Modelo K-Means

Coeficiente de Silueta

El coeficiente de silueta mide qué tan similar es un objeto a su propio cluster en comparación con otros clusters. Los valores oscilan entre -1 y 1, donde valores cercanos a 1 indican una buena agrupación.



Ventajas de K-Means



Simple y eficiente

Algoritmo intuitivo con implementación sencilla y rápida convergencia en la mayoría de los casos.



Escalable a grandes volúmenes de datos

Puede procesar conjuntos de datos extensos con un costo computacional razonable.

Desventajas de K-Means



Requiere definir K previamente

Es necesario especificar el número de clusters antes de ejecutar el algoritmo.

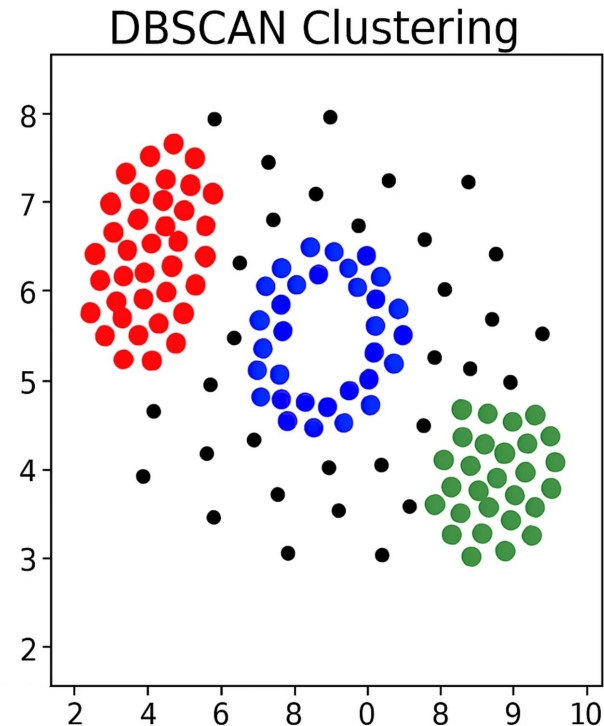


Limitaciones con formas no esféricas

No funciona bien con formas no esféricas o con outliers que pueden distorsionar los centroides.

Algoritmo DBSCAN

DBSCAN se basa en la densidad para formar clusters. Es útil para encontrar estructuras arbitrarias y detectar outliers.



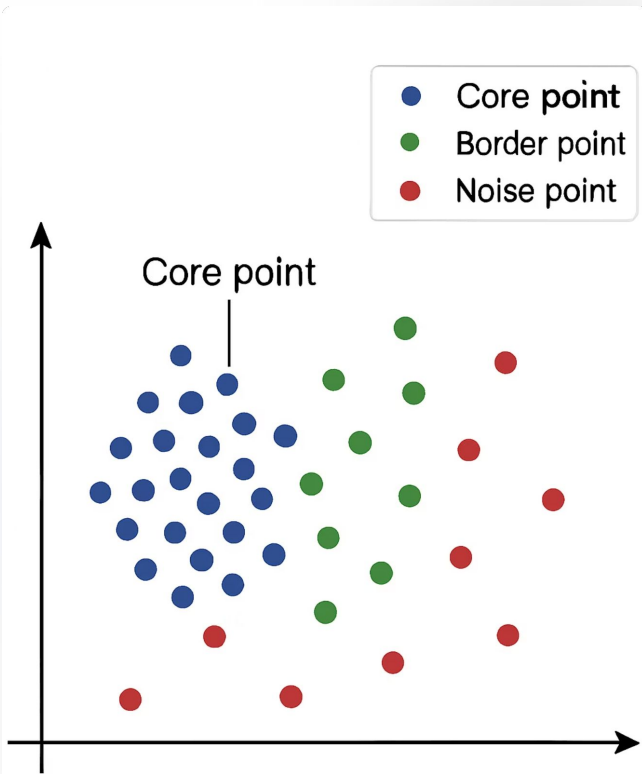
Parámetros Clave de DBSCAN

eps

Radio de vecindad que define la distancia máxima entre dos puntos para considerarlos vecinos.

min_samples

Número mínimo de puntos requeridos en la vecindad de un punto para considerarlo como punto núcleo.



Funcionamiento de DBSCAN

Clasificación de Puntos

Clasifica los puntos como núcleo (core), frontera (border) o ruido (noise) según su densidad de vecindad.

Expansión de Clusters

Expande los clusters desde los puntos núcleo, conectando puntos que son alcanzables por densidad.

Implementación de DBSCAN en Python

```
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

db = DBSCAN(eps=0.5, min_samples=5)
db.fit(X_scaled)

labels = db.labels_
print(set(labels)) # -1 representa ruido
```

Ventajas de DBSCAN



No necesita número de clusters

Determina automáticamente el número de clusters basándose en la densidad de los datos.



Identifica clusters de forma arbitraria

Puede detectar grupos con formas no convencionales que otros algoritmos no logran identificar.



Resistente a outliers

Los clasifica como ruido sin permitir que distorsionen la estructura de los clusters.

Desventajas de DBSCAN



Sensible a la elección de parámetros

Los resultados pueden variar significativamente según los valores de `eps` y `min_samples` seleccionados.



Disminuye su rendimiento en alta dimensión

La eficacia del algoritmo se reduce cuando trabaja con datos de muchas dimensiones debido a la "maldición de la dimensionalidad".

Aplicaciones Prácticas

El uso de algoritmos de clusterización en Python permite implementar soluciones prácticas a problemas del mundo real, desde la segmentación de clientes hasta la detección de patrones en datos masivos.

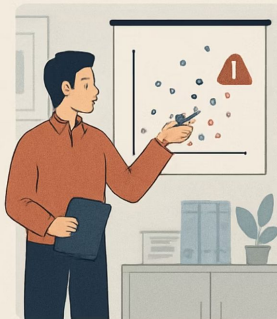
CUSTOMER SEGMENTATION



MARKET SEGMENTATION

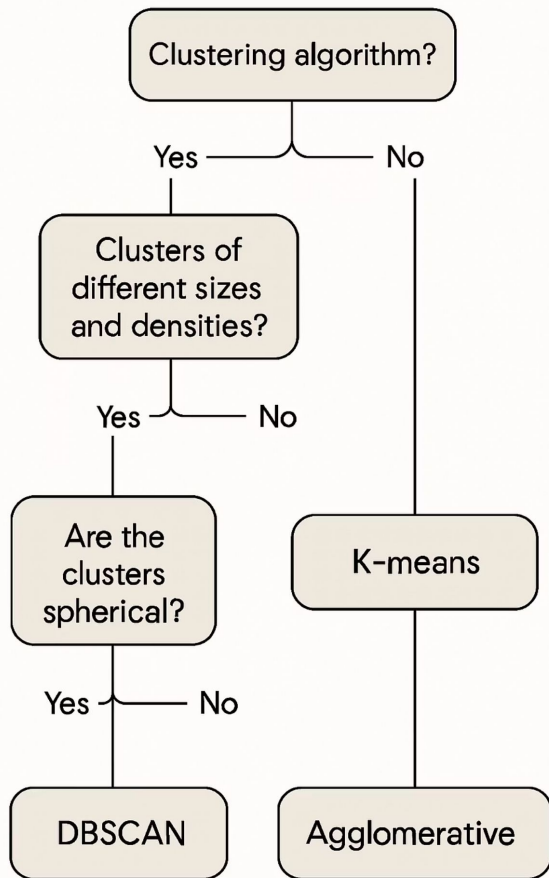


ANOMALY DETECTION



RECOMMENDER SYSTEMS





Selección del Algoritmo Adecuado

Comprender las diferencias entre agrupamiento jerárquico, K-Means y DBSCAN permite seleccionar el método más adecuado según la naturaleza del conjunto de datos y los objetivos del análisis.



Live Coding

¿En qué consistirá la Demo?

Se desarrollará en Python la implementación completa de tres algoritmos de clusterización. El objetivo es poner en práctica la teoría aprendida y analizar los resultados obtenidos con distintos métodos.

1. Preparación del entorno: importación de librerías y dataset sintético
2. Aplicación de K-Means con visualización y análisis del "método del codo"
3. Aplicación de Clustering Jerárquico con generación del dendrograma
4. Aplicación de DBSCAN con ajuste de parámetros eps y min_samples
5. Evaluación con coeficiente de silueta y visualización comparativa
6. Discusión: ¿qué algoritmo funcionó mejor?, ¿qué observamos?, ¿qué limitaciones surgieron?



Momento:

Time-out!

 5 -10 min.





Ejercicio

Resolviendo con clusterización

Resolviendo con clusterización

Contexto: 🙌

Te entregan un conjunto de datos sin etiquetas que contiene múltiples variables relacionadas con comportamiento de usuarios. Debes agrupar a los usuarios según patrones de similitud aplicando diferentes algoritmos de clusterización y comparar los resultados obtenidos.

Consigna: 📝

1. Carga el dataset de ejemplo y realiza el preprocesamiento necesario
2. Aplica K-Means, Jerárquico y DBSCAN con distintos parámetros
3. Visualiza los clústeres obtenidos en cada caso
4. Calcula el coeficiente de silueta para evaluar cada modelo
5. Elige cuál de los tres algoritmos resultó más adecuado y justifica por qué
6. Concluye con una reflexión sobre cómo esta agrupación puede usarse para un modelo predictivo posterior

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Implementamos los algoritmos de clusterización en Python
- ✓ Exploramos el funcionamiento de K-Means, DBSCAN y Jerárquico
- ✓ Aplicamos métricas para evaluar agrupamientos (silueta, método del codo)
- ✓ Analizamos visualmente cómo se comportan con distintos tipos de datos



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

